

Sunrise Dental Clinic — Appointment and Patient Management System

CIS6003 Advanced Programming — WRIT1

Student ID: **st20360306**

ICBT Campus / Cardiff Metropolitan University

Repository: <https://github.com/OGGY404/sunrise-dental-clinic>

Table of contents

1. Introduction	2
2. Task A — System design with UML	3
2.1 Use case model	3
2.2 Class model	6
2.3 Sequence models	8
2.4 Documented assumptions	11
2.5 Critical evaluation of the design	11
3. Task B — Development	11
3.1 Three-tier architecture	11
3.2 Distributed application with web services	12
3.3 Design patterns	12
3.4 The database and its advanced features	14
3.5 Validation, sessions and security	15
3.6 The user interface	16
3.7 Additional functionality delivered	18
4. Task C — Testing	19
4.1 Rationale for the testing approach	19
4.2 How test-driven development was used	20
4.3 Test plan and test data	21
4.4 Test automation and results	22

4.5 Traceability matrix	23
4.6 Evaluation and lessons learned	24
5. Task D — Git and GitHub	24
5.1 Repository and branching strategy	24
5.2 Commits across multiple days	25
5.3 Continuous integration and deployment	26
6. Conclusion	27
References	27
Appendices	28

1. Introduction

Sunrise Dental Clinic is a private dental centre in Colombo whose appointments and records were kept on paper. The scenario names three consequences — double bookings, lost records and billing errors — but they are one fault appearing three times: a paper diary cannot enforce a rule.

This report describes the computerised replacement: a three-tier Java application on Spring Boot 3 and MySQL 8, with a REST web service layer and seventeen Thymeleaf screens. It delivers the six functional requirements and thirteen of the additional features the brief invites, supported by 268 automated tests, eight UML diagrams and a pipeline that builds, tests and releases it. The four assessed parts follow in turn; all source, diagrams and documentation are in the repository above.

2. Task A — System design with UML

2.1 Use case model

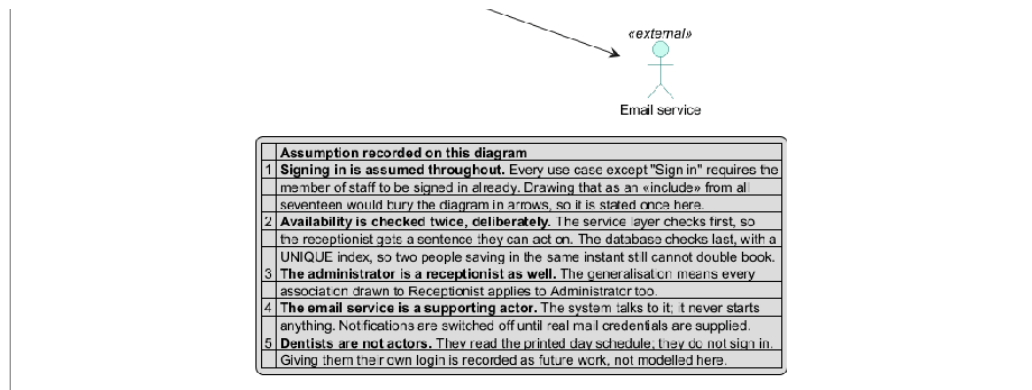


Figure 1: Use case diagram.

Two human actors use the system. The **Receptionist** does the day-to-day work, and the **Administrator** is drawn as a generalisation of the Receptionist, because an administrator does everything a receptionist does and also sees the management reports; the generalisation means every association need only be drawn once.

The **email service** is a supporting actor — the system talks to it, it never starts anything. Dentists are deliberately *not* actors: they read the printed day schedule and do not sign in. Giving them a login is recorded as future work rather than modelled as something that exists.

The <<include>> and <<extend>> stereotypes are used in their proper senses, which are opposites and are commonly confused.

- An **include** is behaviour the base use case performs *every single time*, factored out because more than one use case needs it. *Register appointment* includes *Find or create patient record*: there is no booking without a patient. The arrow points away from the base use case.
- An **extend** happens *only sometimes*, when a condition holds, and the arrow points back *towards* the base use case, which does not know the extension exists. *Suggest another time* extends *Register appointment*, and only when the chosen slot has just been taken.

Four includes and four extends are modelled, each labelled with its condition.

2.2 Class model

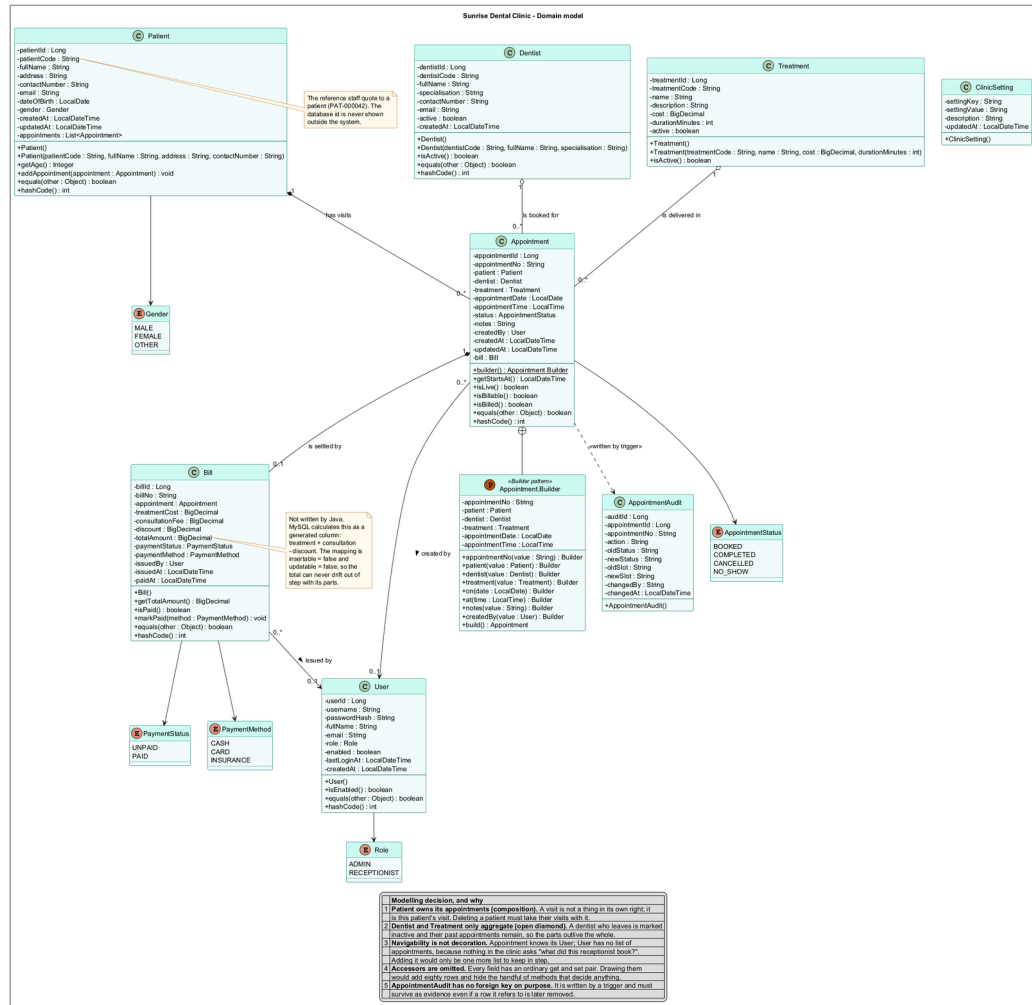


Figure 2: Domain model.

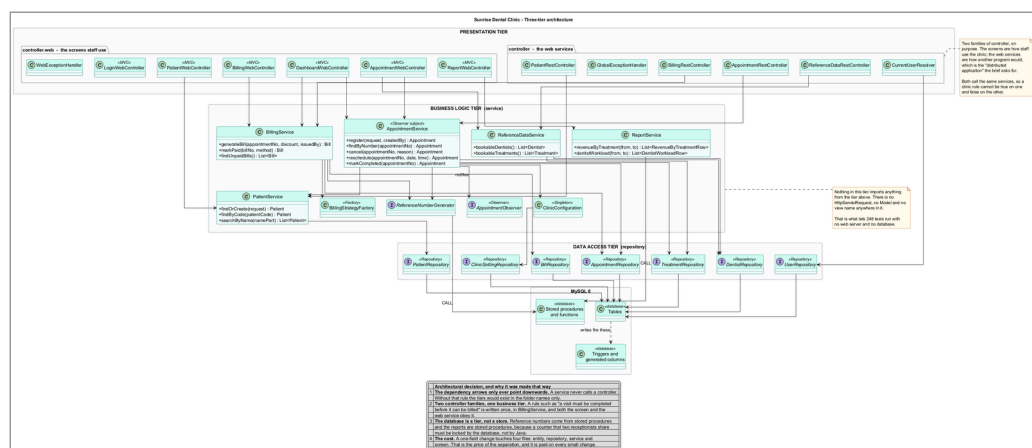


Figure 3: The three tiers and their dependencies.

Every attribute carries its type and every method its full signature, taken from the source rather than remembered, so the diagram and the code agree. All fields are

private with public accessors.

The whole–part relationships are the part of this diagram that required the most thought, and the two kinds are used differently on purpose:

- **Composition** (filled diamond) joins `Patient` to `Appointment`, and `Appointment` to `Bill`. An appointment is not a thing in its own right; it is *this patient's* visit, and it has no meaning if the patient is removed. A bill exists only for the visit it settles.
- **Aggregation** (open diamond) joins `Dentist` and `Treatment` to `Appointment`. The difference is that these parts outlive the whole: a dentist who leaves the clinic is marked inactive rather than deleted, and their past appointments stay in the record permanently.

Multiplicity is shown on every association. In `Appointment 1 ↔ 0..1 Bill` the `0..1` is what allows a visit that has happened but is not yet billed.

Navigability reflects the code rather than decorating the diagram. `Appointment` knows its `User`, but `User` holds no list of appointments, because nothing in the clinic asks "what did this receptionist book?". Figure 3 arranges the same classes by tier, and its single message is that **every dependency arrow points downwards**.

2.3 Sequence models

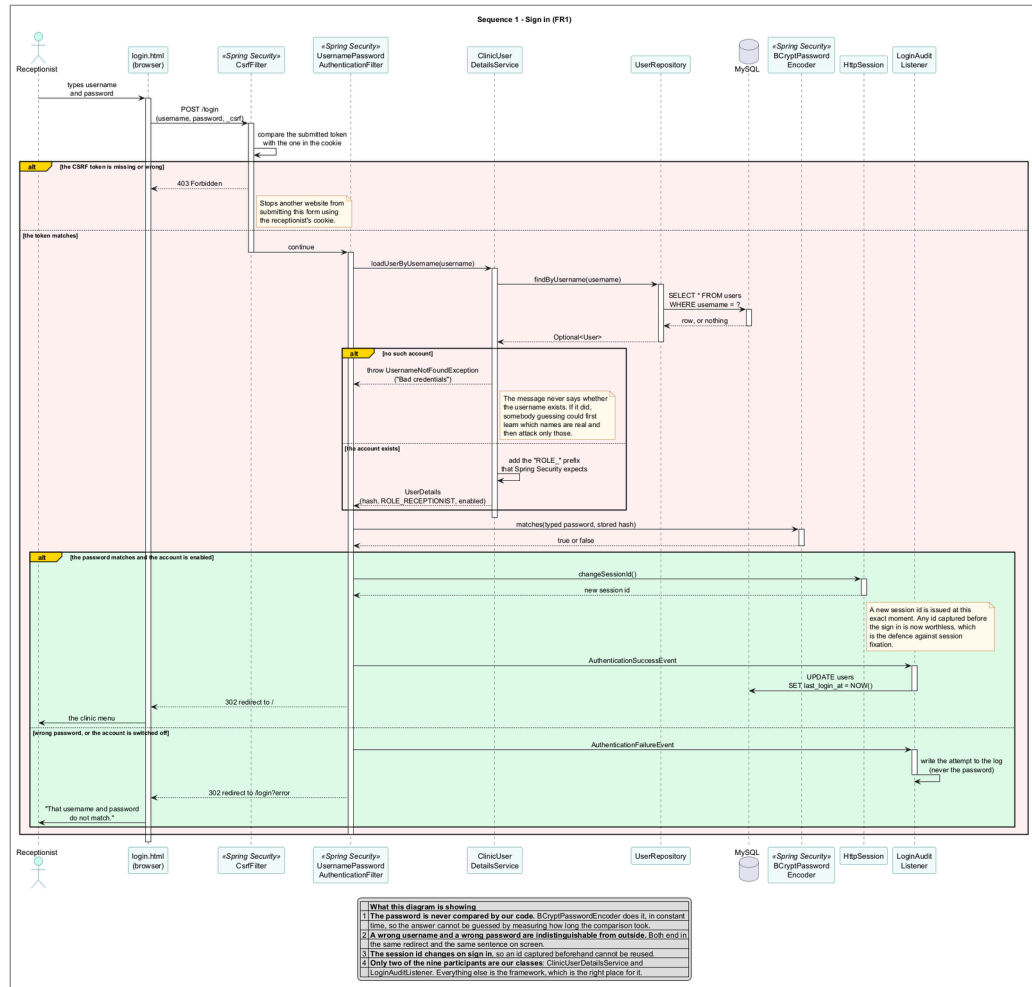


Figure 4: Sequence: signing in (FR1).

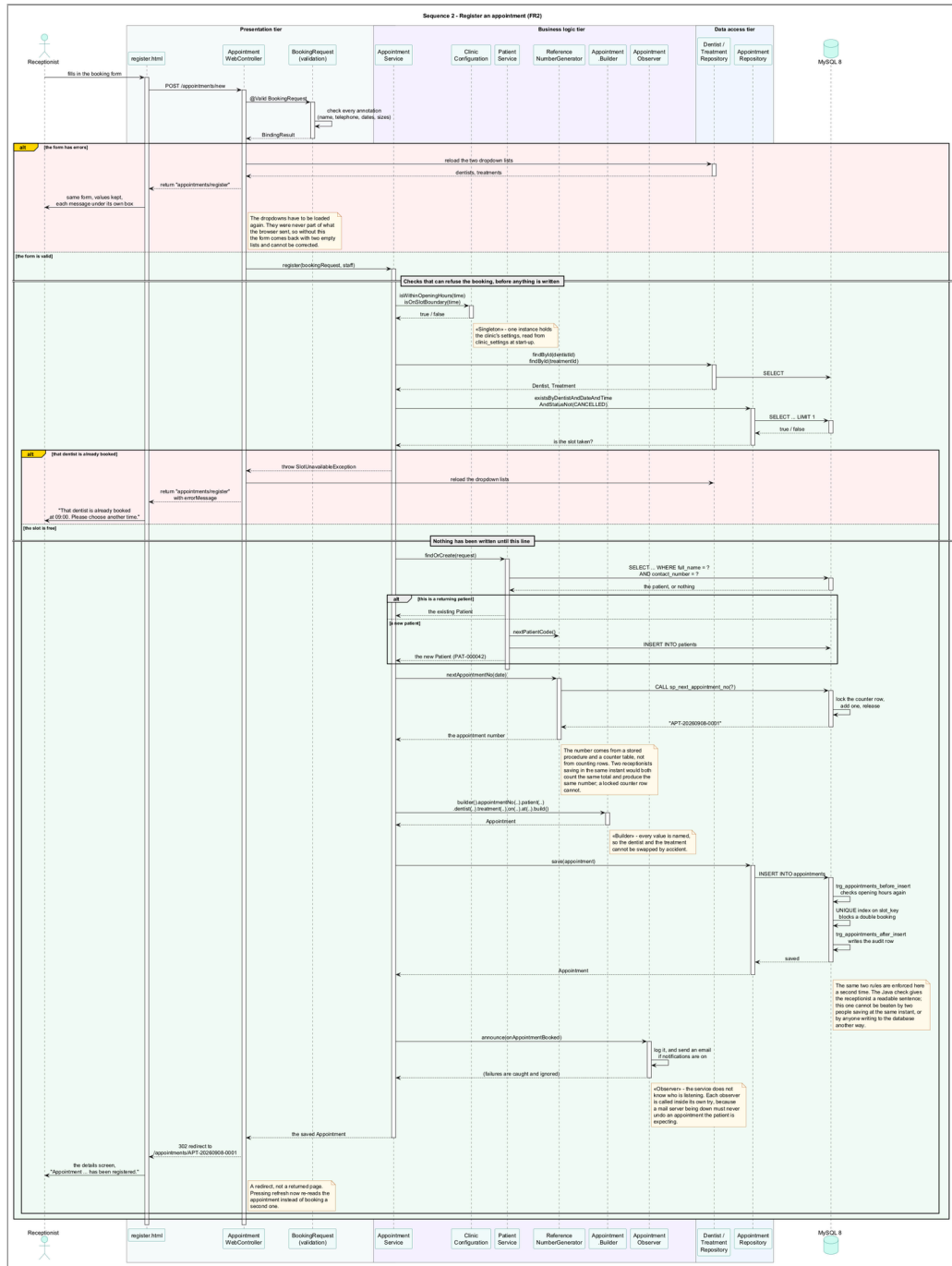


Figure 5: Sequence: registering an appointment (FR2).

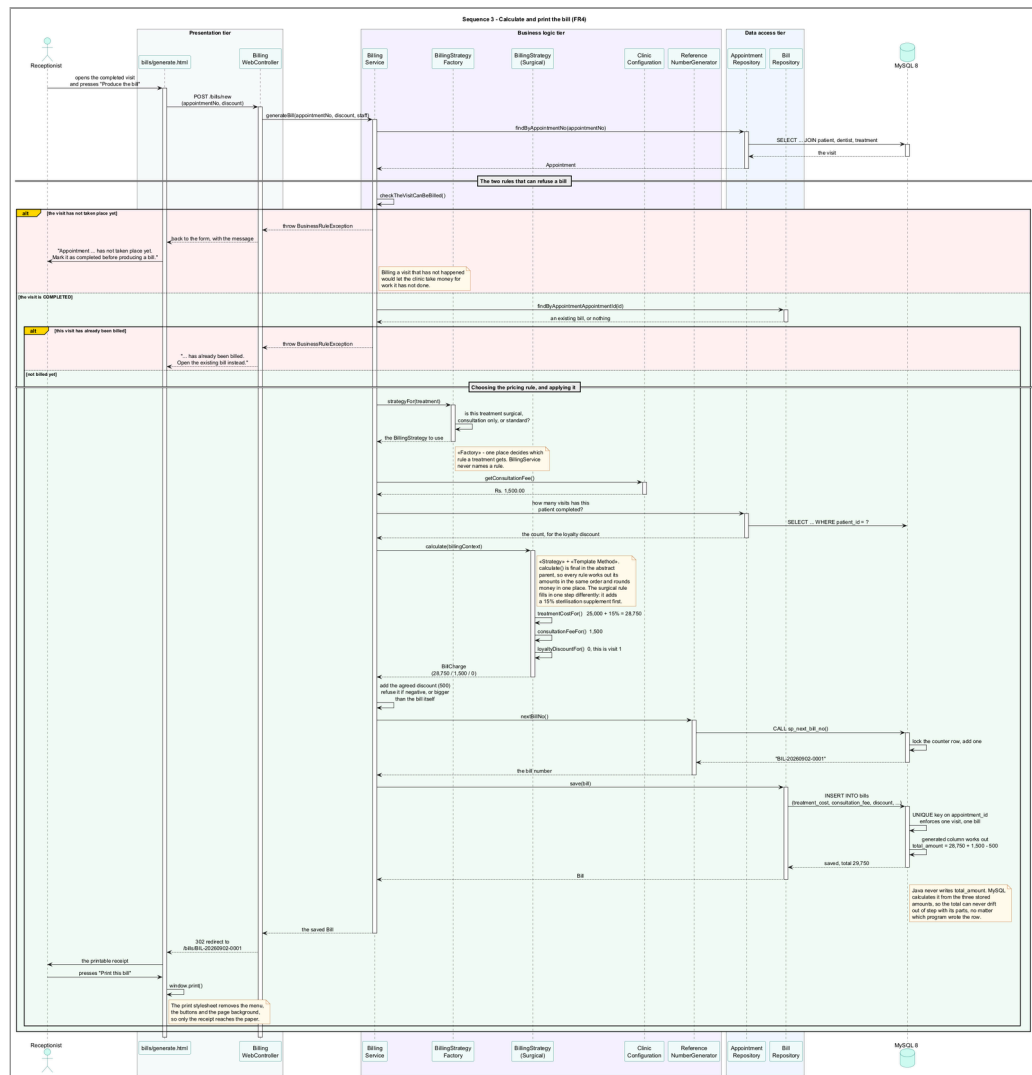


Figure 6: Sequence: calculating the bill (FR4).

Four sequence diagrams were produced; three are shown here and the fourth (cancelling) is in Appendix E. All use proper lifelines, activation bars and `alt` fragments. Three points are worth more than narrating the arrows.

Participants in Figures 5 and 6 are grouped into **tier bands**, and messages only travel downwards and return, so the architecture claimed in Figure 3 is visible as a shape. The `alt` fragments show that the **failure paths were designed rather than discovered**: Figure 5 has three endings — invalid form, slot gone, booking succeeds — each drawn before any code was written. Finally, the note "*Nothing has been written until this line*" marks a deliberate ordering: every check that can refuse a booking runs before any row is created, so a rejected booking cannot leave a half-created patient behind.

2.4 Documented assumptions

Seventeen assumptions were recorded during modelling; the full list is in Appendix D. The five that most affect the design are that patients never use the system themselves; that one dentist sees one patient at a time, so dentist plus date plus time is unique; that a returning patient is recognised by name **and** telephone number, because a name alone is shared by two people and a number alone is shared by a family; that one visit yields one bill, with prices copied onto it at the time of issue so an old receipt survives a price rise; and that cancelling frees the slot while a no-show does not, the dentist's time having already been spent.

Two further assumptions are honest limitations rather than decisions. A long treatment does **not** yet block the following slot, although each treatment records its duration; and several rules are enforced twice, once in Java and again in SQL, which is discussed in §2.5.

2.5 Critical evaluation of the design

The three diagram types agree, and I checked one path to be sure. *Register appointment* is a use case in Figure 1, its classes are in Figures 2 and 3, and its messages are in Figure 5. Where they first disagreed was navigability: my early class diagram gave `User` a list of appointments, but no sequence diagram ever travelled that way, so I removed it.

The hardest decision was enforcing several rules twice, in Java and again in SQL. I kept both. The Java check gives the receptionist a sentence they can act on; the database check cannot be beaten by two people saving in the same instant. The cost is real — the rule exists in two places and could drift apart — but I would do it again, because a double booking hurts the clinic more than a duplicated rule hurts me.

The design is weakest on treatment duration. Each treatment records how long it takes, but a long one does not yet block the following slot. I found this while drawing the class diagram and left it rather than half-build it.

3. Task B — Development

3.1 Three-tier architecture

The code is divided into `controller`, `service` and `repository` packages, with `model`, `dto` and `config` alongside, and one rule holds throughout: **dependencies point**

downwards only. There is no `HttpServletRequest`, no `Model` and no view name anywhere in the business tier.

The evidence is not the folder names but the test suite: **262 of the 268 tests run with no web server and no database at all**, which is only possible because the tiers are genuinely separate. Of the 92 classes in `src/main`, the business tier holds six services, three billing strategies, a factory, four notification classes and a configuration holder — responsibility spread out rather than one large class.

3.2 Distributed application with web services

Seventeen REST endpoints form the web service layer, which is what makes this a distributed application rather than a monolithic desktop program.

They cover registering, displaying, cancelling, rescheduling and completing a visit; patient lookup and treatment history; producing, reading and settling a bill; and the two reference lists the booking form needs. The full table is in Appendix I.

There are deliberately **two families of controller** over one business tier: `controller` serves JSON to programs, and `controller.web` serves HTML to staff. Both call the same services, so a rule such as *"a visit must be completed before it can be billed"* is written once and cannot be true on one interface and false on the other.

3.3 Design patterns

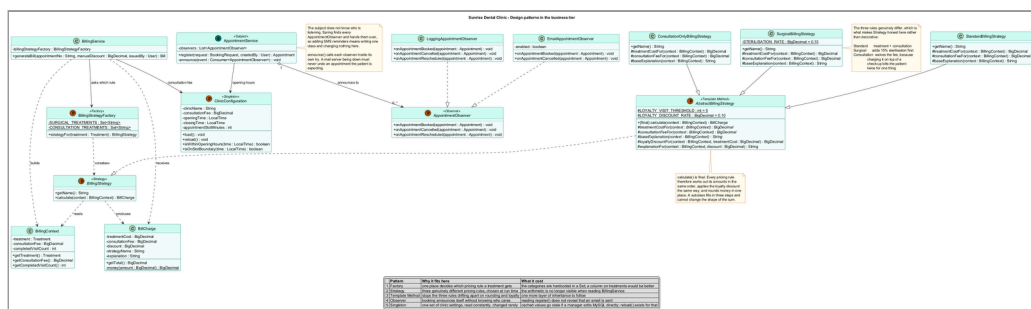


Figure 7: Design patterns in the business tier.

Eight patterns are implemented, each named in a comment in the code and recorded below with what it improved **and what it cost**, because a pattern with no cost has usually not been thought about.

Pattern	Where	What it improved	What it cost
---------	-------	------------------	--------------

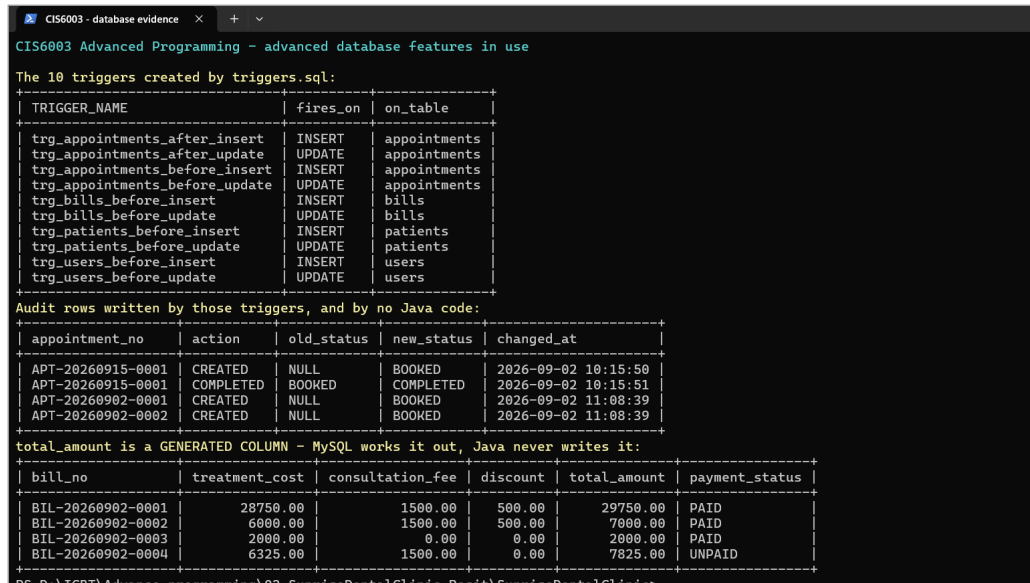
Singleton	<code>ClinicConfiguration</code>	one instance of the clinic's settings, read constantly and changed rarely	cached values go stale if a manager edits MySQL directly; <code>reload()</code> exists for that
Strategy	three billing rules	pricing chosen at run time without <code>BillingService</code> naming any rule	the arithmetic is no longer visible when reading <code>BillingService</code>
Template Method	<code>AbstractBillingStrategy</code>	<code>calculate()</code> is <code>final</code> , so all three rules work in the same order and round money in one place	one more layer of inheritance to follow
Factory	<code>BillingStrategyFactory</code>	one place decides which rule a treatment gets	treatment categories are hardcoded in a <code>Set</code> ; a database column would be better
Builder	<code>Appointment.Builder</code>	every value is named, so dentist and treatment cannot be swapped by accident	about 40 extra lines, and a second place to update
Observer	<code>AppointmentObserver</code> , <code>ReminderNotifier</code>	booking announces itself without knowing who listens; adding SMS means one new class	reading <code>register()</code> does not reveal that an email is sent
Repository	seven Spring Data interfaces	data access declared rather than written	the generated SQL is not visible in the code
MVC	<code>controller.web</code> + Thymeleaf	screens separated from rules	two controller families to keep in step

The Strategy pattern is worth one further sentence, because it is the one most often added decoratively. The three rules here **genuinely differ**:

- **Standard** — treatment cost plus consultation fee.
- **Surgical** — adds a 15 % sterilisation supplement to the treatment cost first (extraction, root canal, wisdom tooth).
- **Consultation only** — waives the consultation fee, because charging it on top of a check-up bills the patient twice for the same thing.

A single method with an `if` could have done this, but each branch would then have had to remember the loyalty discount and the rounding for itself.

3.4 The database and its advanced features



The screenshot shows a terminal window titled 'CIS6003 - database evidence'. It displays the output of a SQL script titled 'CIS6003 Advanced Programming - advanced database features in use'. The output is divided into three sections: 'The 10 triggers created by triggers.sql:', 'Audit rows written by those triggers, and by no Java code:', and 'total_amount is a GENERATED COLUMN - MySQL works it out, Java never writes it:'. Each section contains a table of data.

TRIGGER_NAME	fires_on	on_table
trg_appointments_after_insert	INSERT	appointments
trg_appointments_after_update	UPDATE	appointments
trg_appointments_before_insert	INSERT	appointments
trg_appointments_before_update	UPDATE	appointments
trg_bills_before_insert	INSERT	bills
trg_bills_before_update	UPDATE	bills
trg_patients_before_insert	INSERT	patients
trg_patients_before_update	UPDATE	patients
trg_users_before_insert	INSERT	users
trg_users_before_update	UPDATE	users

appointment_no	action	old_status	new_status	changed_at
APT-20260915-0001	CREATED	NULL	BOOKED	2026-09-02 10:15:50
APT-20260915-0001	COMPLETED	BOOKED	COMPLETED	2026-09-02 10:15:51
APT-20260902-0001	CREATED	NULL	BOOKED	2026-09-02 11:08:39
APT-20260902-0002	CREATED	NULL	BOOKED	2026-09-02 11:08:39

bill_no	treatment_cost	consultation_fee	discount	total_amount	payment_status
BIL-20260902-0001	28750.00	1500.00	500.00	29750.00	PAID
BIL-20260902-0002	6000.00	1500.00	500.00	7000.00	PAID
BIL-20260902-0003	2000.00	0.00	0.00	2000.00	PAID
BIL-20260902-0004	6325.00	1500.00	0.00	7825.00	UNPAID

Figure 8: Triggers, audit rows and the generated column.

The schema is 1,750 lines of SQL across four scripts containing **10 tables, 6 stored functions, 13 stored procedures and 10 triggers**, all created automatically on first start. Four features do work the Java deliberately does not.

Reference numbers come from stored procedures. Counting existing rows and adding one is wrong the moment two receptionists save at the same instant: both count the same total and both produce APT-20260908-0007. `sp_next_appointment_no` locks a single counter row for the fraction of a second it needs, so they get 0007 and 0008. No Java outside the database can give that guarantee.

Double booking is prevented by a generated column. `appointments.slot_key` holds `dentistId|date|time` for a live appointment and `NULL` for a cancelled one, under a `UNIQUE` index. Because SQL permits many `NULL`s in a unique index, cancelling frees the slot **without deleting anything**, and the visit stays in the patient's history.

The bill total is calculated by MySQL. `bills.total_amount` is a generated column computing `treatment_cost + consultation_fee - discount`, mapped `insertable = false, updatable = false` so Java cannot write it. It therefore cannot drift out of step with its parts, whichever program wrote the row.

Triggers keep a record nobody can quietly edit. Ten triggers validate and audit; two of them write to `appointment_audit`, which no Java code ever writes to, so the clinic can answer *"who gave that slot away, and when?"* — the question paper could never answer.

Five stored procedures are called by the running system rather than merely shipped with it: two for reference numbers and three for the reports and reminders, so grouping and totalling happen where the data already is.

3.5 Validation, sessions and security

Sunrise Dental Clinic
Colombo

Home New appointment Find appointment Day schedule Patients Unpaid bills Help reception RECEPTION Sign out

Register an appointment

Every visit gets its own appointment number. A patient who has been here before is recognised from their name and telephone number, so their treatment history stays in one place.

The patient

Full name
Kamal 123
A patient name can only contain letters, spaces, dots, hyphens and apostrophes.

Contact number — 0771234567 or +94771234567
077
Please enter a valid Sri Lankan telephone number, for example 0771234567.

Address
No. 42, Galle Road

Email — optional
kamal@example.lk

Date of birth — optional
dd-....-yyyy

Gender — optional
Not recorded

The visit

Dentist
Please choose a dentist
Please choose a dentist.

Treatment
Tooth Filling - Rs. 6,000.00
The consultation fee is added to this when the bill is produced.

Date
dd-....-yyyy
Please choose a date for the appointment.

Time — the clinic is open 08:00 to 18:00, in half hours
:-:--
Please choose a time for the appointment.

Notes — optional
For example: complains of pain in the lower left molar

Register the appointment Cancel

Sunrise Dental Clinic - Appointment and Patient Management System - C#6003 Advanced Programming

Figure 9: A rejected booking form.

Bean Validation annotations guard every incoming field, and every limit matches the column width in `schema.sql`, so anything that validates can always be stored. A telephone number is accepted as 0771234567 or +94771234567 and in no other shape; a name may hold letters, spaces, dots, hyphens and apostrophes but never digits — the pattern allows W.A.G.K. Rathnayake-Silva, which an earlier letters-only version wrongly rejected.

Figure 9 shows a refusal: the same form returns with every value kept, a message under each offending box, and both dropdowns reloaded. A redirect or a blank form would make the receptionist type everything again.

Sessions and cookies. Login state is kept in a session cookie named `SUNRISEID`, marked `HttpOnly` so page scripts cannot read it and `SameSite=Lax` so another site cannot make the browser send it, timing out after thirty minutes — which matters at a

desk where the screen is left unattended. A new session id is issued at login, so one captured beforehand is worthless afterwards.

Cross-site request forgery protection is left on, because keeping login state in a cookie is precisely what makes that attack possible. Reads need no token; every write does.

Passwords are BCrypt hashes at strength 10, compared by `BCryptPasswordEncoder` in constant time rather than by any code written here. A wrong username and a wrong password are refused identically, so nobody can learn which usernames exist. **Role-based access** separates ADMIN from RECEPTIONIST: the management reports show clinic-wide takings, and a receptionist asking for them receives 403.

3.6 The user interface

The screenshot shows the main menu of the Sunrise Dental Clinic. The header includes the clinic name, location (Colombo), and navigation links: Home, New appointment, Find appointment, Day schedule, Patients, Unpaid bills, and Help. A user is logged in as 'reception' with a 'Sign out' button.

Good day, reception
Today is Wednesday, 2 September 2026

Summary cards:

- 6 appointments booked for today
- 1 bills not yet paid
- Rs. 7,825.00 still owed to the clinic

What would you like to do?

Action cards:

- Register an appointment**: Book a visit for a new or returning patient.
- Find an appointment**: Look a visit up by its appointment number.
- Day schedule**: The diary for one day, ready to print.
- Patients**: Search a patient and read their treatment history.
- Unpaid bills**: What is still owed, oldest first.
- Help**: Step-by-step instructions for new staff.

Today's diary (6)

TIME	APPOINTMENT	PATIENT	DENTIST	TREATMENT	STATUS
09:00	APT-20260902-0001	Kamal Silva	Dr. Nimal Perera	Tooth Filling	COMPLETED
10:00	APT-20260902-0002	Nadeeka Fernando	Dr. Shanika Fernando	Braces Consultation	BOOKED
11:00	APT-20260902-0003	Kamal Silva	Dr. Nimal Perera	Dental Check-up	COMPLETED
14:00	APT-20260902-0004	Ruwan Jayawardena	Dr. Ruwan Jayasinghe	Tooth Extraction	COMPLETED
15:30	APT-20260902-0005	Anusha Perera	Dr. Anusha Wickrama	Child Dental Care	BOOKED
16:00	APT-20260902-0006	Saman Kumara	Dr. Nimal Perera	Scaling and Polishing	BOOKED

Sunrise Dental Clinic - Appointment and Patient Management System - CIS6003 Advanced Programming

Figure 10: The main menu.

Sunrise Dental Clinic

Colombo

[Home](#)
[New appointment](#)
[Find appointment](#)
[Day schedule](#)
[Patients](#)
[Unpaid bills](#)
[Help](#)

[reception](#)
[RECEPTION](#)
[Sign out](#)

APT-20260902-0002

BOOKED

registered on 2 Sept 2026, 11:08

The visit

DATE	TIME	DENTIST	TREATMENT
Wednesday, 2 September 2026	10:00	Dr. Shanika Fernando	Braces Consultation
TREATMENT COST			
Rs. 3,500.00			
Notes			
Wants advice about braces			

The patient

PATIENT NUMBER	NAME	CONTACT NUMBER	ADDRESS
PAT-000002	Nadeeka Fernando	0715558899	No. 7, Temple Road, Nugegoda

The patient has arrived

Mark the visit as completed

The patient did not arrive

Move this appointment

New date

02-Sep-2026

New time

10:00

Move the appointment

Cancel this appointment

The slot becomes free for another patient straight away. A reason has to be given, because it is written into the record.

Reason

For example: the patient telephoned to cancel

Cancel the appointment

Find another appointment

Back to the menu

Sunrise Dental Clinic - Appointment and Patient Management System - CIS6003 Advanced Programming

Figure 11: Appointment details for a booked visit.

Sunrise Dental Clinic

Colombo, Sri Lanka

PATIENT BILL

BILL NUMBER	ISSUED
BIL-20260902-0002	2 Sept 2026, 11:08
PATIENT	APPOINTMENT
Kamal Silva	APT-20260902-0001

Tooth Filling	Rs. 6,000.00
Consultation fee	Rs. 1,500.00
Less discount	- Rs. 500.00
Total	Rs. 7,000.00

PAID

paid by CASH on 2 Sept 2026, 11:08

Thank you for visiting Sunrise Dental Clinic.

This bill was produced by the clinic system and needs no signature.

Figure 12: The bill as printed.

Seventeen Thymeleaf screens share one stylesheet and one navigation bar, so every job is one click away — the menu-driven system the brief asks for. The home screen (Figure 10) also shows the two numbers the front desk needs first thing: patients coming today, and money owed.

Data entry and viewing results are separate screens, as the assessment requires: registering uses `appointments/register` and the result is shown by `appointments/view`; billing uses `bills/generate` and `bills/view`. This is not a formality — every save **redirects** to the result page, so pressing refresh re-reads the record instead of booking a second appointment.

Figure 12 is a print-media render, not the screen with the menu cropped away: the print stylesheet leaves only the receipt, $6,000 + 1,500 - 500 = \text{Rs. } 7,000.00$.

The system also provides a step-by-step **help section** (FR5) and a **sign-out** control (FR6) implemented as a POST carrying the CSRF token.

3.7 Additional functionality delivered

Beyond FR1–FR6, thirteen additional features were built, including cancellation and rescheduling, treatment history, double-booking prevention, role-based access, three reports, an unpaid bills list and appointment reminders.

Sunrise Dental Clinic Colombo

Home New appointment Find appointment Day schedule Reminders Patients Unpaid bills Help reception RECEPTION Sign out

Appointment reminders

Patients still booked for **Thursday, 3 September 2026**. The clinic also sends this round automatically each evening at six.

3 patients due in

2 can be emailed

1 need telephoning

Which day
Tomorrow Show that day

Due in on 3 Sept 2026

TIME	APPOINTMENT	PATIENT	TELEPHONE	DENTIST	TREATMENT	HOW WE REACH THEM
09:00	APT-20260903-0001	Chamari Wijesinghe	0771230001	Dr. Nimal Perera	Tooth Filling	chamari@example.lk
10:30	APT-20260903-0002	Dinesh Rajapaksa	0771230002	Dr. Shanika Fernando	Scaling and Polishing	telephone them
14:00	APT-20260903-0003	Ishara Gunawardena	0771230003	Dr. Ruwan Jayasinghe	Tooth Extraction	ishara@example.lk

Send the reminders now
This does not wait for the evening job. Patients with an email address are emailed; the rest are listed above for you to telephone.

Send the reminders

Print this list Back to the menu

Sunrise Dental Clinic - Appointment and Patient Management System - C156003 Advanced Programming

Figure 13: Appointment reminders.

The reminder feature (Figure 13) reports two counts rather than one: patients emailed, and patients with **no** email address. "Five reminders sent" would be a comfortable statement if two of them could not be reached at all. The split shows the receptionist who still has to be telephoned, which is the only part of the job a computer cannot finish. A scheduled job sends the round each evening.

4. Task C — Testing

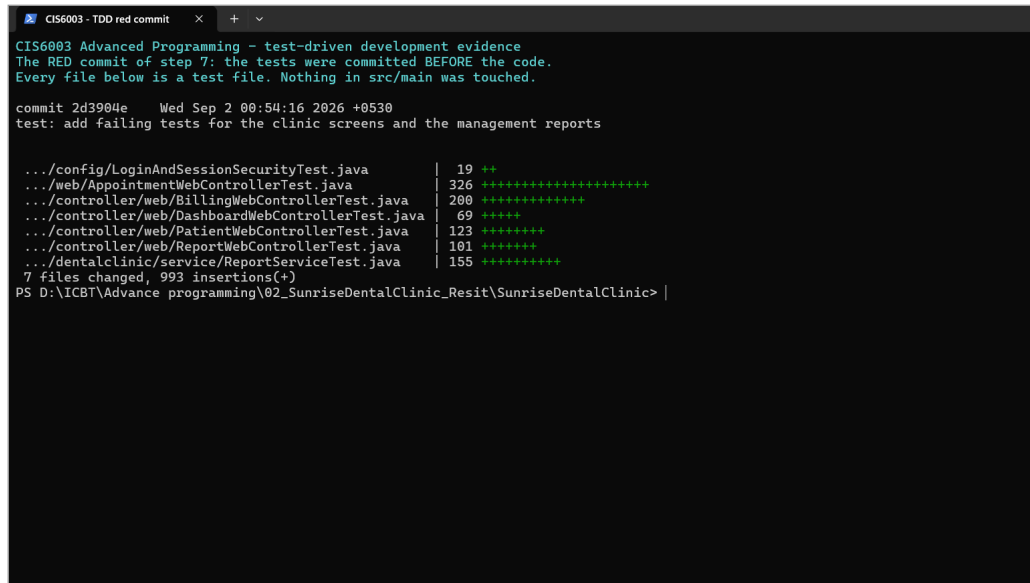
4.1 Rationale for the testing approach

Level	What it tests	Count	Runs against	Speed
Unit	one class, collaborators mocked	123	nothing	milliseconds
Repository	JPA mappings and derived queries	38	in-memory H2	~1 s
Web slice	one controller, services mocked	82	nothing	~1 s
Integration	the whole application context	19	H2, and a real port	~10 s
Database	schema, procedures, triggers	6	real MySQL 8	~10 s

I put most of the tests where they run fastest. The clinic rules change most often, so 123 unit tests sit at the level that reports a mistake in under a minute and needs nothing installed. That speed is why I was willing to run the suite after every small change, and a suite I avoid running is worth very little.

Six database tests looks thin beside 123, but the count is not the point. They cover what no other level can see at all, and each is slow because it starts a real MySQL. Adding more would buy little and would make the suite slow enough that I stopped running it — the failure I was trying to avoid.

4.2 How test-driven development was used



```
CIS6003 - TDD red commit
CIS6003 Advanced Programming - test-driven development evidence
The RED commit of step 7: the tests were committed BEFORE the code.
Every file below is a test file. Nothing in src/main was touched.

commit 2d3904e Wed Sep 2 00:54:16 2026 +0530
test: add failing tests for the clinic screens and the management reports

.../config/LoginAndSessionSecurityTest.java | 19 ++
.../web/AppointmentWebControllerTest.java | 326 +++++
.../controller/web/BillingWebControllerTest.java | 200 +++++
.../controller/web/DashboardWebControllerTest.java | 69 +++++
.../controller/web/PatientWebControllerTest.java | 123 +++++
.../controller/web/ReportWebControllerTest.java | 101 +++++
.../dentalclinic/service/ReportServiceTest.java | 155 +++++
7 files changed, 993 insertions(+)
PS D:\ICBT\Advance programming\02_SunriseDentalClinic_Resit\SunriseDentalClinic> |
```

Figure 14: A red commit: only test files.

TDD is not claimed here; it is visible in the Git history. On **seven separate occasions across four days**, the failing tests were committed *before* the code that makes them pass. Each red commit deliberately does not compile, because the classes its tests name do not exist yet.

Step	Red commit	Green commit	Tests after
Domain model	5c5c357	a05f7c4	39
Design patterns	2e89427	dab3032	77
Service layer	4fa3ebc	49c5559	121
Web services	ca70acc	1e07179	183
Security	a41b34b	2e068c6	205
The screens	2d3904e	ee29194	248
Reminders	22e09fc	7c8b027	262

Any of them can be verified:

```
git show 2d3904e --stat      # only test files
git checkout 2d3904e
mvnw test-compile           # fails: cannot find symbol
AppointmentWebController
```

Refactoring — the step most often left out — is also recorded. The three pricing rules were first written as three independent classes and all their tests passed. Reading them

afterwards showed the same three steps repeated in each, so `calculate()` was pulled up into an abstract parent and made `final`. **No test changed.** That is what made the refactor safe, and it is precisely what the tests were for.

Two decisions the red step changed are worth naming. Writing `todayIsAllowed` before the annotation forced the question "is today past or future?" to be answered deliberately — `@Future` would have been the obvious choice and would have been wrong, because a patient with toothache can walk in and be seen that afternoon. Writing `aFailingObserverDoesNotLoseTheBooking` first is why each observer is called inside its own `try`.

4.3 Test plan and test data

The full test plan is in Appendix B: 111 cases, each with an ID, precondition, steps, test data, expected and actual result, pass/fail, and the name of the automated test that runs it. Every case is automated; none was run by hand and then written down.

Test data was **derived** from the specification — column widths, opening hours, the rules in the brief — and **devised** where a specific risk needed probing, using equivalence partitioning and boundary value analysis throughout.

Examples are tabulated in Appendix B: a name of 101 characters against the 100-character column, a telephone number one digit short, yesterday against today, and the fourth against the fifth completed visit for the loyalty rule.

The most valuable case is TC-DB-05, which inserts a duplicate slot straight into the table with `JdbcTemplate`, skipping every Java check. It is the only way to prove that a rule really lives in the database rather than only in the application.

4.4 Test automation and results

```
"D:\ICBT\Advance programmi...
CIS6003 Advanced Programming - Sunrise Dental Clinic
Running the full automated test suite: .\mvnw.cmd test

2026-09-02T22:42:35.024+05:30 INFO 16380 --- [Sunrise Dental Clinic] [main] l.i.c.d.service.ReminderService
s) ahead: There are no appointments booked for that day, so no reminders were needed.
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.203 s -- in lk.icbt.cis6003.dentalclinic.service.Remin
rs
[INFO] Running lk.icbt.cis6003.dentalclinic.service.ReminderServiceTest$ReadingTheList
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in lk.icbt.cis6003.dentalclinic.service.Remin
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.208 s -- in lk.icbt.cis6003.dentalclinic.service.Remin
[INFO] Running lk.icbt.cis6003.dentalclinic.service.ReportServiceTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.030 s -- in lk.icbt.cis6003.dentalclinic.service.Repor
[INFO] Results:
[INFO]
[INFO] Tests run: 262, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ dental-clinic ---
[INFO] Loading execution data file D:\ICBT\Advance programming\02_SunriseDentalClinic_Resit\SunriseDentalClinic\target\jacoco.e
[INFO] Analyzed bundle 'Sunrise Dental Clinic' with 75 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 28.789 s
[INFO] Finished at: 2026-09-02T22:42:35+05:30
[INFO]
PS D:\ICBT\Advance programming\02_SunriseDentalClinic_Resit\SunriseDentalClinic> |
```

Figure 15: The full suite: 262 tests, 0 failures.

Sunrise Dental Clinic										
Sessions										
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes		
lk.icbt.cis6003.dentalclinic.model	66%	34%	86	234	105	388	57	199	0	14
lk.icbt.cis6003.dentalclinic.service	83%	70%	40	115	47	301	14	68	0	6
lk.icbt.cis6003.dentalclinic.service.notification	48%	50%	10	18	37	71	7	15	1	3
lk.icbt.cis6003.dentalclinic.controller.web	85%	73%	17	63	19	179	9	46	0	7
lk.icbt.cis6003.dentalclinic.controller	78%	66%	11	48	24	113	6	39	0	6
lk.icbt.cis6003.dentalclinic.service.billing	85%	75%	7	49	7	84	2	38	0	7
lk.icbt.cis6003.dentalclinic.config	94%	80%	6	57	7	140	1	44	0	6
lk.icbt.cis6003.dentalclinic.dto	97%	0%	2	106	3	186	1	105	0	13
lk.icbt.cis6003.dentalclinic.exception	84%	n/a	1	7	2	13	1	7	0	5
lk.icbt.cis6003.dentalclinic	37%	n/a	1	2	2	3	1	2	0	1
Total	1,079 of 5,639	80%	104 of 270	61%	181	698	253	1,478	99	563

Created with JaCoCo 0.8.12.202403310830

Figure 16: JaCoCo coverage report.

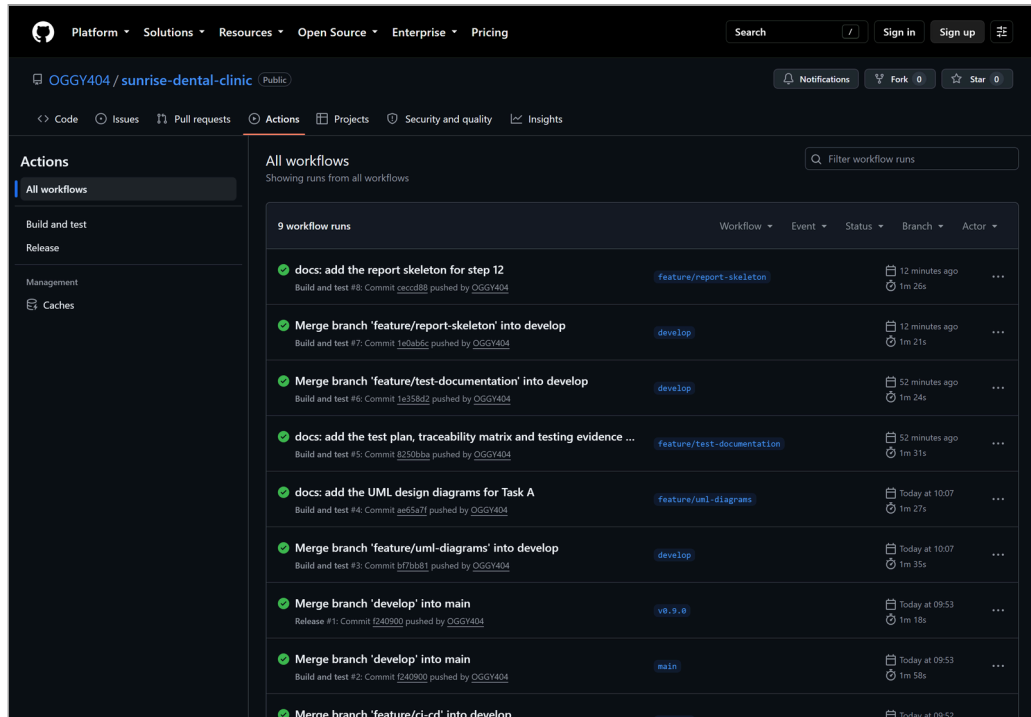


Figure 17: GitHub Actions: the three CI jobs.

268 automated tests across 32 test classes, with no failures. JaCoCo reports **78.8 % instruction** and **62.2 % branch coverage**. Everything runs from one command and needs no human: `ci.yml` runs the whole suite on every push, repeats the database half against a real MySQL 8 container, then packages the jar — and only if both test jobs passed.

Two figures are deliberately low. `model` sits at 66.7 % because most of it is accessors no test calls directly, while the methods that decide anything are covered; `service.notification` sits at 32.6 % because email is switched off until real credentials exist, and raising it would mean mocking a mail server to prove Spring can send an email.

4.5 Traceability matrix

Appendix C traces every requirement from the design that models it, through the code that implements it, to the tests that verify it. A requirement with an empty test column would be one nobody has checked; there are none. The six core requirements are summarised here:

FR1 is covered by 9 cases, FR2 by 30, FR3 by 6, FR4 by 20, and FR5 and FR6 by one and two respectively; the thirteen FR7 features add a further 44.

Three items are **not** automated, and the matrix says so: printing on paper, sending a real email, and the visual appearance of the screens. A matrix claiming complete coverage would not be true.

4.6 Evaluation and lessons learned

The suite did not catch everything, and that is the most useful thing I learned. Three bugs reached the running application while every test was passing.

Bug	Why every test passed
<code>password_hash</code> was <code>CHAR(60)</code> in SQL but <code>VARCHAR</code> on the entity, so the application would not start against MySQL	H2 builds its tables from the entities , so the two could never disagree
The CSRF cookie was deleted at login and never reissued, so every save after signing in returned 403	MockMvc supplies its own token instead of reading the cookie
The tables and the database used different collations, so a stored function could not compare its parameter against a column and every booking failed	H2 has no stored functions at all

The cause is the same in all three: both H2 and MockMvc were supplying one side of the exchange and then checking the other, so the suite confirmed its own assumptions rather than testing them.

I added two levels in response — six tests against a real MySQL, and two driving a real HTTP port with real cookies — and checked both by removing the fix and watching them fail, which is the only way to know a regression test works. If I did this again I would start the database tests at step 2, when the schema was written, rather than at step 9; an hour then would have saved all three bugs.

On balance TDD was worth it, but not for the reason I expected. It caught fewer bugs than I assumed, and instead forced decisions to be made deliberately: writing `todayIsAllowed` before the annotation is why a walk-in can be booked today, where `@Future` would have been the obvious choice and wrong.

5. Task D — Git and GitHub

5.1 Repository and branching strategy

The public repository is <https://github.com/OGGY404/sunrise-dental-clinic>.

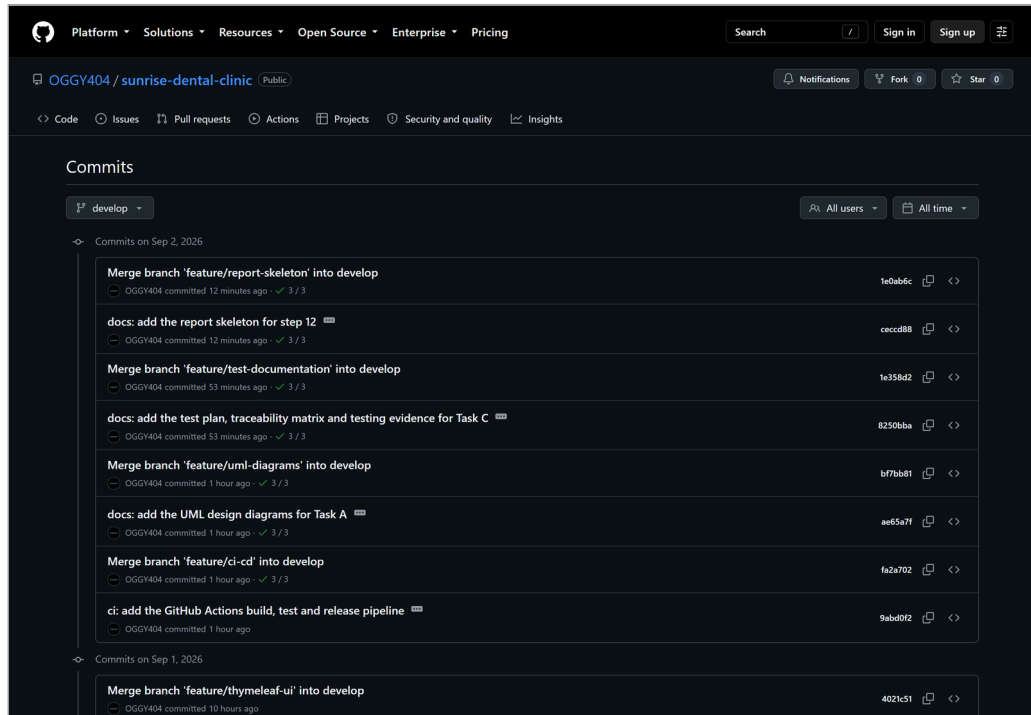


Figure 18: Commit history.

A three-level branching strategy was used:

- **main** holds released versions only, and carries the tags.
- **develop** is the integration branch.
- ****feature/**** — eleven branches, one per step of the build, from database-layer through to appointment-reminders.

Every feature branch was merged with `--no-ff`, so the history shows the shape of the work rather than flattening it into one line.

5.2 Commits across multiple days

42 commits across four separate days, each adding a feature incrementally: 6 on 21 August (repository, schema, procedures, triggers), 9 on 28 August (entities, patterns, services), 7 on 1 September (web services, validation, security) and 20 on 2 September (screens, reports, CI/CD, UML, documentation and reminders).

Messages follow the conventional style — `feat:`, `fix:`, `test:`, `ci:`, `docs:` — and each explains *why* rather than restating the diff. The `test:` commits deliberately precede their `feat:` counterparts, which is the TDD evidence in §4.2.

A `.gitignore` keeps `target/`, IDE folders and `.env` out of the repository, so no credential has ever been committed. `.env.example` is committed instead, as a

template.

5.3 Continuous integration and deployment

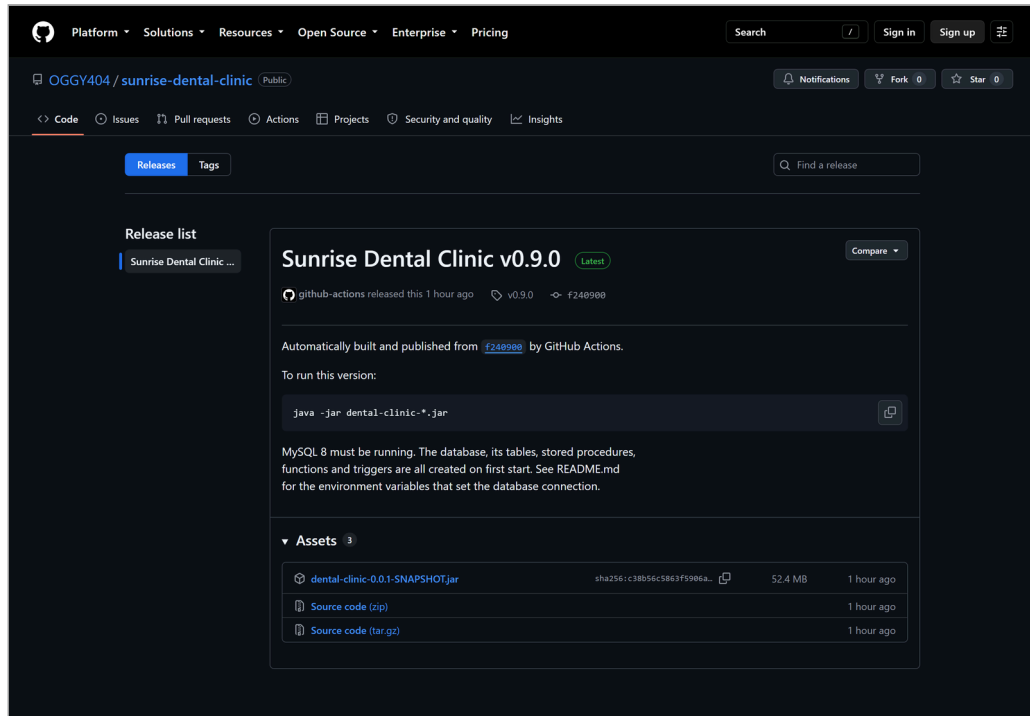


Figure 19: Release v0.9.0 with the runnable jar.

`ci.yml` runs three jobs in order on every push and pull request: the 262 H2 tests, then the same application against a real MySQL 8 service container, then packaging the jar — and only if both test jobs passed. The second job exists because of the bugs in §4.6: H2 can only agree with the entities it was built from, and has none of the stored procedures, triggers or generated columns.

`release.yml` handles deployment. Pushing a version tag builds from a clean checkout, **runs the tests again** — a tag can be pushed from any machine, and a released version must never be one nobody proved — then publishes the runnable jar as a GitHub Release. **v0.9.0** is published with the jar attached (Figure 19). Because Spring Boot packages the web server inside the jar, deployment is Java 17, MySQL 8 and one command:

```
export DB_HOST=localhost DB_NAME=sunrise_dental DB_USERNAME=clinic
DB_PASSWORD=...
export COOKIE_SECURE=true # once served over HTTPS
java -jar dental-clinic-0.0.1-SNAPSHOT.jar
```

The database, tables, procedures, functions and triggers are all created on first start, so a new server needs nothing set up by hand.

6. Conclusion

The system delivers all six functional requirements and thirteen additional features, supported by 268 automated tests, eight UML diagrams and a pipeline that builds, tests and publishes a runnable release.

I am most pleased that the double-booking rule lives in the database: two receptionists pressing Save in the same instant cannot break it, and no Java outside the database could have guaranteed that. What I would change is the treatment duration limitation, and starting the real database tests far earlier.

The lesson I take forward is narrower than "write tests": a test only tells you something when it can disagree with you.

References

(Excluded from the word count. Cite only what you actually read — a list containing books you have not opened is itself an offence, and markers ask.)

Candidates, if you read them:

- Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
- Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley.
- Beck, K. (2002) *Test-Driven Development: By Example*. Boston, MA: Addison-Wesley.
- VMware (2024) *Spring Framework Reference Documentation*. Available at: <https://docs.spring.io/spring-framework/reference/> (Accessed: DATE).
- VMware (2024) *Spring Security Reference*. Available at: <https://docs.spring.io/spring-security/reference/> (Accessed: DATE).
- Oracle (2024) *MySQL 8.4 Reference Manual*. Available at: <https://dev.mysql.com/doc/refman/8.4/en/> (Accessed: DATE).
- OWASP (2024) *Cross-Site Request Forgery Prevention Cheat Sheet*. Available at: <https://cheatsheetseries.owasp.org/> (Accessed: DATE).

Appendices

(Excluded from the word count — use them generously.)

Appendix	Contents	Source
A	Source code listings of the key classes	src/main/java/...
B	Full test plan, 111 cases	docs/testing/test-plan.md
C	Full traceability matrix	docs/testing/traceability-matrix.md
D	All 17 documented assumptions	docs/uml/README.md
E	Sequence diagram 4 — cancel an appointment	docs/uml/png/08-sequence-cancel-appointment.png
F	Remaining screenshots	docs/report/screenshots/
G	End-to-end walkthrough output	docs/testing/manual-walkthrough-output.txt
H	Database scripts	src/main/resources/db/*.sql
I	Full list of the 17 web service endpoints	README.md